

FEBAUTO Racing

Organiser manual

Part I The platform and the rules

Part II Running the programme: tracks, events, race day

Part III Maintaining the platform: builds, releases, reference

Formula Electric at Berkeley

September 2026, platform release v0.1.0

https://github.com/FEBAutonomous/feb-racing

https://febautonomous.github.io/feb-racing/

Contents

I The platform 2

1 What it is 2

2 Rules 3

II Running the programme 4

3 The organiser PC 4

4 Tracks 4

5 Events 5

6 Race day: time-attack 6

7 Finals: head-to-head 6

8 Between events 7

9 A semester 7

10 Checklist for the first run-through 7

III Maintaining the platform 7

11 What reaches members, and how 7

12 Building and releasing the simulator 8

13 Devkit image 9

14 Timing and comparability 9

15 Frequently hit 10

16 Command reference 10

17 Repository layout 10

Part I
The platform

1 What it is

FEBAUTO Racing is a sim-racing programme for autonomous driving. Members write a ROS 2 driving stack for a 1:10 scale RoboRacer car, run it in the FEB Simulator on their own laptop, and post lap times to a public leaderboard. Twice a semester the organiser runs a competition on a secret track, with the same rules as the RoboRacer Sim Racing League.

The simulator is a fork of the AutoDRIVE Simulator (Unity). The car, its sensors and its physics are the league's RoboRacer model, unchanged, so a stack that works here works in the league too.

1.1 The pieces

Piece	What it does	Who touches it
FEB Simulator	Native app (Mac, Windows, Linux). Simulates the car on a track folder and talks to a bridge over a WebSocket on port 4567.	Everyone runs it, nobody edits it
Devkit container	Docker image `ghcr.io/febautonomous/feb-devkit`: ROS 2 Humble plus the AutoDRIVE bridge that turns the WebSocket into ROS 2 topics. The member's code runs inside it.	Members' code runs inside it
`stack/`	The member's ROS 2 packages. Mounted into the container and built on every start.	Members
`tracks/`	Public tracks. PNG and YAML in, `track.json` out. Shipped inside the app.	Organiser adds, members use
`feb`	The member launcher: setup, up, shell, code, practice, submit, reset, update.	Members
`feb-race`	The organiser harness: events, nightly reruns, verification, head-to-head brackets.	Organiser
`results/`, `events/`, `submissions.yaml`	The data behind the leaderboard site.	Harness writes, members' pull requests add
Site	GitHub Pages, rebuilt on every push to `main`.	Read-only

1.2 Data flow while driving

```
simulator --> bridge --> /autodrive/roboracer_1/* --> sim adapter --> /scan /odom /imu /
    camera/color/image_raw --> your node
your node --> /drive --> supervisor --> /ackermann_cmd --> sim adapter --> bridge -->
    simulator
```

On the real car the VESC and lidar drivers stand where the bridge and the adapter are; from the node outward nothing changes. The car takes one input and everything else is a sensor:

- `/drive`, `ackermann_msgs/AckermannDriveStamped`: `drive.speed` in m/s and `drive.steering_angle` in rad, at most ±0.5236. The steering actuator lags about 0.25 s and slews at 3.2 rad/s. A command older than 0.5 s stops the car. The supervisor clips the speed to the mode's cap; the adapter turns it into the simulator's throttle (feed-forward plus a bounded trim, never the hard-brake zero while speed is wanted).

1.3 Sensors

The contract, the real car's topics:

Topic	Type	Notes
`/scan`	`sensor_msgs/LaserScan`	1080 beams over 270°, 0.06 to 10 m, 40 Hz, frame `laser`
`/odom`	`nav_msgs/Odometry`	pose and speed dead-reckoned from the wheel encoders and the gyro, as the VESC does it
`/imu`	`sensor_msgs/Imu`	orientation, angular velocity, linear acceleration
`/camera/color/image_raw`	`sensor_msgs/Image`	forward camera (cone tracks)
`/feb/state`	`std_msgs/String`	the supervisor: IDLE, MANUAL, AUTONOMOUS or STOP, and why

The simulator's own topics (namespace `/autodrive/roboracer_1/`), for debugging only:

Topic	Type	Notes
`lidar`, `imu`, `front_camera`	raw sensors	what the adapter republishes
`left_encoder`, `right_encoder`	`sensor_msgs/JointState`	wheel angle in rad; wheel radius 0.059 m
`steering`, `throttle`	`std_msgs/Float32`	actuator feedback
`ips`, `odom`, `tf`	pose	ground truth: **training and debugging only**
`lap_count`, `lap_time`, `last_lap_time`, `best_lap_time`, `collision_count`	counters	scoring: **training and debugging only**
`reset_command`	command	respawn: **training and debugging only**

Wheelbase 0.324 m, half width 0.135 m, steady-state speed about 23 m/s per unit throttle. With a window open the bridge exchanges data at about 10 Hz; headless it reaches 20 to 40 Hz. Scored runs go through a proxy fixed at 10 Hz for everyone, so a driver must cope with 10 Hz. The starter kit does.

1.4 Time

Physics runs in *simulated* time. When the machine cannot keep up (a screen recorder, a browser, two cars) the simulator advances physics by at most 0.1 s per frame, so simulated time falls behind the wall clock rather than the car jumping. Everything is built so that this cannot hurt a driver: every message stamp and the `/clock` topic carry simulated time, the starter kit runs with `use_sim_time`, lap timers count simulated time, the scoring proxy delivers 10 Hz of *simulated* time, and the render rate is capped so it rarely happens at all. The HUD shows the real-time factor (RTF, simulated seconds per wall second; red below 0.9) and every result records it. If you write your own nodes, use the message stamps or `use_sim_time`, never the wall clock, for speeds and integrals.

2 Rules

These follow the RoboRacer Sim Racing League.

- One attempt is 1 warm-up lap plus 10 timed laps. Total time is the sum of the timed laps plus **10 s per collision**.
- Not finishing within **300 s** is DNF, ranked after every finisher by laps completed. More than **10 collisions** is DSQ.
- Each team gets the number of attempts stated in the event; the best counts.
- **Qualifying for the first competition**: on *every* practice track, a verified practice run of 10 laps with at most 1 collision. Practice tracks are added through the semester; the bar applies to all of them. The Teams page of the site shows each team's progress per track.
- **The final** is head-to-head, two cars at a time, in a knockout seeded by the qualification standings; only the top seeds (set per event, 8 by default) enter.
- **Restricted topics at race time**: `ips`, `odom`, `tf`, the lap and collision counters, and `reset_command`. The harness records which nodes subscribe to them during the run and flags the result on the site. Use them freely while developing.
- Everything a submission needs starts from the container's entrypoint (`/home/autodrive_devkit.sh`), never from `.bashrc`.
- Head-to-head: car-to-car contact counts as a collision for both cars and respawns them side by side. Finishing order is most laps first, then the finish time including penalties.

Part II
Running the programme

3 The organiser PC

Everything official runs on the team PC with the headless Linux simulator. Members never need access to it. The organiser works on it through Claude Code sessions in `tmux` (remote desktop through Sunshine and Moonlight when a screen is needed); the commands below are what those sessions run.

Item	Path
Repos	`~/FEB/feb-racing` (main, this manual), `~/FEB/feb-sim` (Unity fork, branch `feb`)
Simulator players	`~/FEB/feb-sim/Builds/{linux,mac,windows}`; Linux also linked at `~/.feb-sim/app/linux`
Unity editor	`~/Unity/Hub/Editor/2022.3.52f1`, licensed through Hub sign-in
Run folders	`~/FEB/feb-racing/runs/<id>/` (`result.json`, `sim.log`, `container.log`, bag)
Secret tracks	`~/feb-secret-tracks/<name>/`, same layout as `tracks/`, never committed until released
Ports	4567 member devkit; 4568+ harness containers, one per car; 4580 harness proxy; 4565/4566 launcher proxy and opponent

4 Tracks

A track is a folder. The simulator reads `track.json` at launch, so adding a track never needs a Unity rebuild; it needs an app release only to ship inside the app.

```
tracks/<name>/
    design.yaml    optional: waypoints + width, rendered to map.png
    map.png        white = drivable corridor, everything else = not drivable
    track.yaml     name, resolution, direction, checkpoints, walls, cones, max_cars
    track.json     GENERATED: walls, centreline, checkpoints, spawn, cones
    preview.png    GENERATED: picture for the site
```

4.1 Create a new track (two commands)

```
mkdir tracks/hairpins
cat > tracks/hairpins/design.yaml <<EOF
width: 2.0
points: [[0, 0], [12, 0], [12, 4], [4, 4], [4, 8], [14, 8], [14, 12], [0, 12]]
EOF
cat > tracks/hairpins/track.yaml <<EOF
name: Hairpins
direction: ccw
checkpoints: 20
walls: {diameter: 0.33}
EOF
tools/track_design.py tracks/hairpins/design.yaml   # map.png, resolution and origin
    into track.yaml
tools/track_build.py tracks/hairpins                # track.json + preview.png
./feb up --track hairpins                           # look at it
```

The corridor is a smooth closed spline through the waypoints: any number, any shape, no self-intersections. Any other PNG works too: hand-painted, or a SLAM occupancy grid of a real track (Porto is one; its resolution and origin come from the ROS `map.yaml`). Grey "unknown" pixels count as not drivable.

`track.yaml` key	Meaning
`name`	shown in the simulator and on the site
`resolution`, `origin`	metres per pixel and map position of the bottom-left pixel (written by `track_design.py`)
`direction`	`ccw` or `cw`
`checkpoints`	lap checkpoints; checkpoint 0 is the finish line
`max_cars`	1 for corridors too narrow to race side by side (Porto); default 4
`qualifying`	`false` for showcase tracks too long for the 300 s attempt (Spielberg, the circuit): left out of qualification and nightly reruns
`start`	optional `[x, y]` finish-line position; default: the straightest section
`walls.diameter`	air-duct diameter, 0.33 m like the league; omit for a cone-only track
`cones.spacing`	metres between cones on each edge, blue left, yellow right; omit for none

4.2 Secret tracks

Design competition tracks in `~/feb-secret-tracks/<name>/` with the same two commands. The event file points at that folder and the harness passes it to the headless simulator directly. After the event, move the folder into `tracks/`, commit, and cut an app release so it ships inside the simulator as a practice track.

5 Events

One YAML per event in `events/`. Pushing it lists the event on the site.

```
id: 2026-10-qualification
title: Fall 2026 Qualification
date: 2026-10-15
mode: time-attack            # or head-to-head
status: upcoming             # upcoming | live | done
track: ~/feb-secret-tracks/qual-2026-10
attempts: 2                  # best attempt counts
rules: {laps: 10, warmup_laps: 1, collision_penalty_s: 10, max_collisions: 10, timeout_s
    : 300}
entries: qualified           # all | qualified | a list of team names
```

`entries: qualified` admits the teams that have a verified practice run of all laps with at most one collision on every practice track (`./feb-race qualified` lists who is there and what each team is missing; the site's Teams page shows the same). A head-to-head event takes `top: 8` to limit the knockout to the top seeds of the event it is seeded from. Adding a practice track raises the bar for everyone, since the rule covers every track in `tracks/`; announce new tracks early.

Teams register their image in `submissions.yaml` by pull request (name, GitHub logins, public image built from the devkit). Review that the image is public and the launch command is set before merging.

6 Race day: time-attack

```
cd ~/FEB/feb-racing
git pull                                    # latest submissions.yaml
./feb-race event 2026-10-qualification      # every registered image, N attempts each
git add results && git commit -m "qualification results" && git push
```

Each attempt: pull the image, start the container, start the headless simulator on the secret track behind the 10 Hz proxy, probe laps and collisions inside the container, record a bag, tear down, write `result.json`. Standings print at the end and the site rebuilds itself on push. Afterwards set `status: done` in the event file and release the track.

One attempt takes about 3 minutes. Ten teams with two attempts is about an hour. `-gui` on any harness command shows the simulator window instead of running headless.

7 Finals: head-to-head

```
./feb-race bracket 2026-12-final --seed-from 2026-10-qualification
./feb-race h2h --teams "Team A" "Team B" --track tracks/loop --laps 10 --gui   # one
    race
```

Seeds come from the time-attack standings: single elimination, 1 v N, 2 v N−1, top seed has a bye on an odd count. Race ids are `r<round>-m<match>`; the site shows every race under the event and ranks teams by the round they reached.

```
simulator --cars 2 --connect :4580  <->  proxy (4580)  <->  container A (4568, sees V1)
                                                       <->  container B (4569, sees V1)
```

Each car runs in its own container behind the proxy (`febrace/proxy.py`), which rewrites the vehicle ids so every container sees its own car as `roboracer_1`, unchanged from time-attack. Cars start in single file, 2.5 m apart, the higher seed in front. Results carry `race`, `opponents`, `finish_s` and `position` (most laps first, then finish time).

The official AutoDRIVE Race Control Tower does the same rewriting and adds stewarding: after any car-to-car contact it freezes both cars until a race director rules in its web UI. That suits a live final with a human steward and not automated brackets, which is why the harness has its

own proxy. Two cars is the supported race mode; three or more works end to end but drivers do not see each other, so contact and pair respawns pile up. Treat it as experimental.

8 Between events

```
./feb-race nightly                                    # rerun every registered image on
    every practice track
./feb-race verify results/practice/loop/<file>.json # rerun one self-reported entry
./feb-race standings 2026-10-qualification
./feb-race site                                       # rebuild site/data.json locally
```

Put `nightly` in cron on the PC and push `results/` afterwards. Self-reported practice results arrive as pull requests that validate and merge themselves (`.github/workflows/results-pr.yml`); the nightly turns them verified.

9 A semester

When	What	Where it shows
Week 0	Install, drive manually, run the starter driver, post one practice result	`loop` practice board
Weeks 1 to 5	Self-paced learning path, one practice track per stage	practice boards, nightly reruns
Mid-semester	Qualification: time-attack on a secret track, 2 attempts	the event page
Weeks 7 to 12	Weekly open rounds on rotating tracks; released tracks join the pool	practice boards
Finals	Time-attack on a new secret track for seeding, then a head-to-head knockout	the event page with races

Roles: the organiser owns the secret tracks, the events folder and the PC; members own their `stack/`, their images and their practice results. Members learn by tuning against a leaderboard; the organiser reviews rather than lectures.

10 Checklist for the first run-through

1. Make the devkit package public on GitHub (Packages, `feb-devkit`, Package settings, Change visibility). Until then `feb setup` fails at the pull for anyone but you.
2. On a member laptop (your Mac): follow the competitor manual exactly as written, from `git clone` to `./feb submit`. Note every step that needs a hint the manual does not give.
3. Design the first secret track, write its event file, push it.
4. Run `./feb-race event` once with the example team to see a full result land on the site.

Part III
Maintaining the platform

11 What reaches members, and how

Members run `./feb update` (or just `git pull`); nothing is pushed to them. The launcher tells them when a newer app exists.

You change	Members get it by	App rebuild?
A practice track in `tracks/`	`git pull`: the launcher passes the repo's tracks folder to the app, so the track picker shows it	no (only to ship it inside the app)
Launcher, harness, docs, rules	`git pull`	no
The devkit image (`devkit/`)	CI publishes it; `feb update` pulls it and replaces their box; their next image build inherits it	no
The simulator itself (`Assets/FEB`)	you build and upload the release; `feb update` replaces the app	yes
Results, events, site	nothing: the site rebuilds on push	no

12 Building and releasing the simulator

The simulator is the `feb-sim` repo: a fork of the AutoDRIVE Simulator (Unity 2022.3.52f1, HDRP, Mono backend) with everything FEB-specific under `Assets/FEB`. The RoboRacer vehicle prefab (`Assets/Prefabs/F1TENTH/F1TENTH.prefab`) and the upstream scripts are untouched. A rebuild is needed only when the app changes or a released track should ship inside it.

```
cd ~/FEB/feb-sim
./compile_check.sh                      # compiles Assets/FEB against the editor, no
    licence needed
./build.sh scene                        # regenerates the FEB scene from the upstream
    scene
./build.sh linux && ./build.sh mac && ./build.sh windows
tar -C Builds/linux --exclude=./runs -czf FEB-Simulator-linux.tar.gz .
(cd Builds/mac && zip -qr ../../FEB-Simulator-mac.zip "FEB Simulator.app")
(cd Builds/windows && zip -qr ../../FEB-Simulator-windows.zip .)
gh release upload v0.1.0 -R FEBAutonomous/feb-racing --clobber FEB-Simulator-*.tar.gz
    FEB-Simulator-*.zip
```

The shipped tracks are copied from `../feb-racing/tracks` at build time. The first build imports the whole project (30 to 90 minutes); later builds take minutes. Logs are in `Logs/`. `feb setup` and `feb update` download from the latest release and remember the asset's upload time, so uploading with `-clobber` to the existing release is enough: members are told a newer app exists the next time they run. Make a new release tag when the launcher's expectations change.

12.1 What the fork adds (`Assets/FEB`)

File	Role
`Scripts/TrackLoader.cs`	builds walls, checkpoints, cones, spawn and cameras from a track folder at runtime; clones the car for `-cars N` (clones keep only their sensor cameras); applies the FEB decals
`Scripts/TrackData.cs`, `TrackLibrary.cs`	the `track.json` model and where tracks are found (StreamingAssets/Tracks, `~/.feb-sim/tracks`)
`Scripts/FebLaunch.cs`, `FebAutoStart.cs`	command-line options, bridge address, starting mode and camera
`Scripts/FebTrackMenu.cs`	the Track and Cars buttons in the menu
`Scripts/GhostLap.cs`	records and replays the best lap as a translucent gold car
`Scripts/FebLook.cs`, `FebHud.cs`	the Visual look (asphalt, banded walls, start line, sky, livery; no colliders, no physics change) and the race HUD
`Editor/FebScene.cs`	generates the FEB scene from the upstream RoboRacer Sim Racing scene
`Editor/FebBuild.cs`	player builds, icon, shipped tracks

Command-line options of the built app (Unity's own `-batchmode -nographics -logFile` also work):

```
--track <name|folder>   --connect <host:port>   --mode manual|autonomous
--lidar-hz <rate>       --cars <n>              --camera <name>         --look visual|
    simple
```

12.2 Unity licence on the build PC

Unity Personal is free for the team but must be activated once per machine. Easiest: start Unity Hub on the PC's screen (via Moonlight), sign in, Installs, Locate, `~/Unity/Hub/Editor/2022.3.52f1/Editor/Unity`, then Preferences, Licenses, add a free personal licence. Without a GUI: generate an activation file with `Unity -batchmode -nographics -quit -createManualActivationFile`, upload it at https://license.unity3d.com/manual, and activate the downloaded `.ulf` with `-manualLicenseFile`. Details in `docs/unity-licence.md`.

13 Devkit image

`devkit/` holds the Dockerfile (ROS 2 Humble, the AutoDRIVE bridge, pinned Python packages) and the entrypoint, which builds every package under `src/` and then starts the bridge and the command in `FEB_LAUNCH`. `devkit/feb_tools` adds the map-following house driver and the sensor-noise node. The entrypoint also starts `foxglove_bridge` on port 8765 (off with `FEB_FOXGLOVE=0`, which the harness and the opponent container set), so members visualise with the Foxglove app on any OS instead of rviz. Pushing a change under `devkit/` rebuilds and publishes `ghcr.io/febautonomous/feb-devkit` for amd64 and arm64 (`.github/workflows/devkit.yml`). Members' images are built *from* this one, so a devkit change reaches them on their next image build.

14 Timing and comparability

Runs on different machines are comparable because the harness scores in simulated time: the simulator stamps every message (`Sim Time`), the bridge publishes `/clock`, the proxy paces the devkit at 10 Hz of simulated time, and lap timers are simulated time. Each `result.json` records `real_time_factor`, the run's simulated seconds per wall second; a value well below 1 means the PC was overloaded during that run (rerun it, and keep recorders, browsers and Foxglove off the organiser PC during events). The simulator caps rendering at 60 fps (`-fps`) and reads cameras

back at 10 Hz (`-camera-hz`) so that physics keeps real time; headless runs need neither. Details: `STATUS.md`, 2026-09-14 timing entry.

15 Frequently hit

Symptom	Cause and fix
No window after a driver update	GPU driver mismatch on the PC: reboot.
Port 4567 in use	A box is running on it (that is normal; practice uses 4667, the harness its own ports). To free it: `./feb reset`.
Attempt shows no laps	The image's stack never started: read `runs/<id>/container.log`. Check `FEB_LAUNCH` and that the packages built.
Result flagged *restricted topics*	A node in the image subscribed to `ips`, `odom`, `tf`, counters or `reset_command`. The audit lists the node names.
Two cars touch at the start	The track's spawn straight is too narrow for the grid; set `max_cars: 1` in its `track.yaml`.
Cones or walls missing	The track folder has no `track.json`; run `tools/track_build.py`.

16 Command reference

Command	What it does
`feb-race event ID`	run a time-attack event
`feb-race bracket ID -seed-from ID`	run a head-to-head knockout
`feb-race h2h -teams A B -track T [-laps] [-gui]`	one race
`feb-race run -image I -team T -track T [-event] [-laps] [-timeout]`	one attempt of one image
`feb-race nightly`	rerun every registered image on every practice track
`feb-race verify FILE`	rerun one self-reported result
`feb-race standings ID`	print an event's standings
`feb-race qualified`	who has qualified, and what each team still lacks
`feb-race site`	rebuild `site/data.json` locally
`feb up -track T -driver D -opponent L -cars N`	the member launcher, also handy for looking at a new track
`tools/track_design.py FOLDER/design.yaml`	waypoints to `map.png`
`tools/track_build.py FOLDER`	`map.png` to `track.json` and `preview.png`

17 Repository layout

```
feb-racing/
    feb, feb-race              launchers (Python 3, no dependencies beyond docker and gh
    )
    febrace/                   harness library: runner, probe, proxy, rules, results,
    site, h2h
    devkit/                    Dockerfile + entrypoint of the devkit image, feb_tools
    starter_kit/               feb_driver package + submission Dockerfile
    tools/                     track_design.py, track_build.py, unity_install.py
    tracks/                    public tracks
```

```
    events/, results/, submissions.yaml, site/    leaderboard data and page
    .github/workflows/         site (Pages), devkit (image), results-pr (self-merging
    results)
    docs/                      handbook, the two manuals, detail pages
    STATUS.md                  engineering log, newest first
feb-sim/  (branch feb)         Unity fork; Assets/FEB holds everything FEB-specific
```